

Article

AI-Driven Approaches to System Requirements and Test Case Generation: A New Paradigm in Software Engineering

Ziad Salem ^{1,*} , Luay Tahat ^{2,*} , Yasmeen Humaidan ¹ and Noor Tahat ¹ 

¹ Computer Science Department, Center for Applied Mathematics & Bioinformatics (CAMB), College of Arts and Sciences, Gulf University for Science and Technology, Mubarak Al-Abdullah, Hawally 32093, Kuwait; gust0022433@gust.edu.kw (Y.H.); gust0025915@gust.edu.kw (N.T.)

² Management Information Systems Department, College of Business Administration, Gulf University for Science and Technology, Mubarak Al-Abdullah, Hawally 32093, Kuwait

* Correspondence: salem.z@gust.edu.kw (Z.S.); tahat.l@gust.edu.kw (L.T.); Tel.: +965-25307329 (Z.S.)

Abstract

Artificial intelligence (AI) is a new paradigm in software engineering that automates key phases of the development cycle. The methods of creating test cases and designing requirements are still mostly manual and prone to error. Unclear requirements can result in expensive rework and undiscovered defects in the development process. Scalability and dependability are crucial concerns in complex systems. These shortcomings highlight the need for improved methods to enhance accuracy and consistency throughout these critical phases. To generate well-organized system requirements, this article outlines a clear strategy that leverages Extended Finite State Machine models as formal inputs for large language models (LLMs). Five system models are used to assess the suggested framework. The comparison analysis evaluates the accuracy, completeness, test coverage, and runtime efficiency of the artifacts. Along with a comparison with a human-made reference standard, the study evaluates the performance of LLMs such as ChatGPT-5, Claude Sonnet 4.5, and DeepSeek V3.2. The findings demonstrate that AI models can achieve human-comparable accuracy by exceeding 90% with EFSM-based prompting. Claude Sonnet generated the most reliable findings, ChatGPT demonstrated exceptional flexibility, and DeepSeek demonstrated exceptional runtime economy. These findings show that human–AI workflows provide a new paradigm in scalable, traceable, and reproducible system engineering.

Keywords: artificial intelligence; software engineering; extended finite state machine; large language models; code generation; test automation; requirements engineering



Academic Editor: Sotirios K. Goudos

Received: 23 March 2026

Revised: 18 April 2026

Accepted: 21 April 2026

Published: 25 April 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

The rapid development of artificial intelligence (AI) has completely transformed the design, development, and validation of software systems. AI-driven components are being used more and more in the software development process, helping to automate some tasks that people usually do [1].

Large language models (LLMs), such as ChatGPT, Claude, and DeepSeek, have become effective software agents in this regard [2]. They can comprehend system-level specifications and produce technical artifacts such as executable source code, unit tests, and natural-language requirements. LLMs have a significant impact on system behavior, quality, and evolution as they are integrated into software engineering workflows and decision-making procedures [3].

Despite their potential, little is known about the accuracy, completeness, and dependability of AI-generated software outputs, especially when compared to formal system models and human-engineered baselines, which refer to manually developed implementations designed using traditional software engineering or control design principles, depending on the system type. The majority of evaluations are still restricted to prompt-based experimentation, single-model analysis, or informal qualitative assessment, despite recent studies demonstrating the promise of generative AI in discrete software engineering tasks [4].

There are still no systematic, model-driven evaluation frameworks that check how well AI-generated products meet the engineering standards and formal behavioral specifications. Specifically, not enough research has been conducted on AI models' capacity to maintain state behavior, guard conditions, and transfer logic from formal system representations to consistent requirements, accurate executable code, and verifiable test cases. The degree of human interaction needed to convert AI outputs into production-ready objects, traceability, and semantic correctness are all seriously threatened by this gap.

This paper shows and demonstrates how structured model inputs, disciplined rapid design, and iterative evaluation can help close the gap between conceptual modeling and practical software by incorporating AI directly into the software engineering workflow. The ultimate objective is to provide a repeatable framework for assessing and enhancing human–AI collaboration in system design, requirement engineering, and test generation rather than to replace engineers.

To assess the performance of various AI models when given the task of producing requirements, code, and test cases from formal Extended Finite State Machine (EFSM) models [5,6], this study presents a structured model-driven framework.

Instead of relying solely on descriptive prompts, we employed EFSM diagrams, textual specifications, and pseudocode to ensure that each model received the same structured inputs. This method enabled a fair and repeatable comparison of model behavior under carefully monitored experimental settings.

Building on prior EFSM-based modeling work, this study extends the scope from a single-case analysis to a multi-model evaluation including ChatGPT, Claude Sonnet, and DeepSeek, as well as a human engineer baseline [7]. EFSMs were used as the main way to model because they clearly show how a system behaves by outlining specific state changes, conditions that must be met, and actions taken. This formal structure makes EFSMs a viable choice for grounding the generation of large language models because it makes sure that the artifacts that come from them are consistent, traceable, and can be systematically verified. Each model was required to generate structured system requirements, executable Python source code, and unit test suites, which were verified using pytest, a widely used testing framework for automated validation.

A requirements document is a structured specification that defines the functional and non-functional needs of a system, including system behavior, constraints, and interactions with external entities. It serves as a foundational artifact in the software development life cycle, guiding design, implementation, and testing activities while ensuring that stakeholder expectations are clearly and consistently captured.

The generated outputs were then evaluated in terms of accuracy, completeness, test coverage, and human editing effort across five representative modules: the A/C Controller, Food Order System, Online Quiz, Temperature Control System, and ATM. Finally, the evaluation results show that using EFSM-based prompts helps LLMs create software artifacts that are very accurate and thorough, reaching up to 90% agreement with human-made standards in the systems studied. These findings indicate that using organized human–AI processes backed by formal behavioral models can create a reliable and expandable method for requirements engineering, code creation, and automated testing.

This study proposes a unified EFSM-driven framework for generating and evaluating three key software engineering artifacts: system requirements, executable code, and test cases. While testing is used as a mechanism to assess behavioral correctness, it represents only one component of a broader multi-artifact evaluation pipeline. The primary contribution of this work lies in integrating formal EFSM models with large language models to enable structured, consistent, and reproducible generation across multiple stages of the software development life cycle.

The remainder of this article is organized as follows. Section 2 reviews related work on AI-assisted requirements engineering, code generation, and model-based testing. Section 3 explains the basic concepts of EFSM-based system modeling that are used as formal inputs for creating artifacts. Section 4 describes the proposed methodology and comparative study design. Section 5 presents the experimental setup and reproducibility controls. Section 6 defines the evaluation metrics used to assess generated artifacts. Section 7 reports the results across all systems and evaluated AI models. Section 8 discusses key findings, cross-model insights, and implications for software engineering practice. Section 9 discusses the threats to the validity of the study. Finally, Section 10 concludes the article. The Appendix provides additional EFSM diagrams and other model systems requirement specifications.

2. Related Work

The application of AI within software engineering has attracted significant scholarly attention, particularly in areas related to requirements engineering, code generation, and automated software testing. Rather than focusing solely on the novelty of AI itself, recent research has increasingly examined how AI can be systematically integrated into structured development workflows to improve accuracy, consistency, and productivity throughout the software development life cycle (SDLC).

Early work in this area laid the groundwork for modern AI-based implementations by exploring methods for automatically generating test cases from structured specifications [8]. This approach highlighted how structured inputs can enhance the efficiency and reliability of test case generation, forming the basis for later AI-driven techniques.

Other work examined AI techniques in system testing, concluding that hybrid human–AI pipelines consistently outperform manual testing strategies in terms of coverage and time efficiency [3]. Further work complemented this study with a systematic mapping of requirements engineering practices tailored for AI-enabled systems, emphasizing transparency, explainability, and ethical oversight [1].

The role of AI within the requirements engineering community has been discussed, emphasizing that intelligent assistants can enhance specification completeness and consistency when trained on formal modeling data [2].

Similarly, a tertiary study was conducted that synthesized research trends in AI for requirements engineering, showing a marked shift from purely descriptive natural-language approaches toward model-driven and hybrid frameworks [4]. This topic was further expanded by mapping techniques for generating executable test cases directly from formalized requirements, reinforcing the connection between system models and validation processes [9].

In the domain of model transformation and validation, various frameworks were developed to verify EFSM models—an approach closely aligned with the modeling methodology adopted in this research [7]. The findings show that EFSM-based representations allow for clear and traceable changes from design documents to testable code, making them excellent starting points for AI-assisted generation processes.

Recent studies have gone beyond traditional software engineering to look at how generative models affect modern development practices. The role of large-scale LLMs

across SDLC was analyzed, identifying their potential to automate requirement writing, code synthesis, and documentation tasks with minimal human intervention [10,11].

One of the first studies conducted looked at how ChatGPT helped in software engineering, showing that it could make the process faster and improve the completeness of requirements but still emphasized the necessity for humans to check for context and ethical issues [12].

Together, these studies established the conceptual and technological foundation for this research. The literature reveals a consistent movement toward AI-assisted model-driven engineering, where formal models such as EFSMs guide the automatic derivation of requirements, code, and tests. However, it also underscores a persistent gap in cross-model comparative evaluation, specifically how different LLMs perform when exposed to identical structured input.

While prior studies on AI-assisted requirements engineering, code completion tools, and automated test case generation have demonstrated significant progress, most of this research focuses on individual phases of the SDLC in isolation. Additionally, many assessments are restricted to testing one model at a time, using casual observations or simple examples without linking the results to formal behavior guidelines. In contrast, this work presents a combined framework that uses executable system models as organized inputs to systematically create requirements, source code, and test suites all in one clear process. Additionally, this study offers one of the first controlled comparisons of several top LLMs against a human-engineered standard across different typical systems. By focusing on making results repeatable, comparing different models, and using formal models as a foundation, this work goes further than current methods and creates a flexible way to include AI in careful software engineering processes.

3. EFSM-Based System Modeling

Structured formal inputs are still sorely needed to ensure the correctness, consistency, and repeatability of artifacts created by AI, building on previous research that demonstrates how AI is rapidly being used to automate requirements, code, and test generation. To fill this need, this section presents EFSM models [5,6], which provide organized inputs for AI to create requirements, test cases, and source code.

State-based system models describe the behavior of the system using a collection of states and their transitions. The most popular formal description techniques for describing state-based systems are state diagrams, specification description language (SDL), and EFSMs [5,13]. Because these languages are often graphic, they are easy to use and understand. They are frequently used in business, particularly in fields like computer networking, embedded systems, and telecommunications, where state-based systems are prevalent.

In this study, the EFSM model serves as the formal design abstraction, enabling the generation of requirements, code, and test cases within a model-driven engineering paradigm.

This work focuses on system models expressed as EFSMs, which are the formalism that forms the basis of other state-based modeling languages, such as SDL and Statecharts. An EFSM consists of a finite set of states, including a start state and an exit state, together with transitions triggered by events when their associated enabling conditions evaluate to true. Each transition performs a predefined set of actions that may produce output or update variables, thereby altering the system state.

Formally, an EFSM M is expressed as a 7-tuple $M = (\Sigma, Q, \text{Start}, \text{Exit}, V, O, R)$, where Σ is the set of events, Q is the set of states, $\text{Start} \in Q$ is the start state, $\text{Exit} \in Q$ is the exit state, V is a finite set of variables, O is the set of actions, and R is the set of transitions.

Each transition T is represented by the tuple $T = (E, C, A, S_b, S_e)$, where E denotes the triggering event, C is an enabling condition (guard) defined over V , A is a sequence of

actions $A = (a_1, a_2, \dots, a_j)$ where $a_i \in O$, the state $S_b \in Q$ is the transition's originating state, and $S_e \in Q$ is the transition's terminating state [5]. Furthermore, the following transition T-related notation is presented:

For transition $t \in T$, the originating (source) state is denoted by $S_b(t)$, and its destination (ending) state is denoted by $S_e(t)$. The enabling condition, expressed by a Boolean expression, is represented by $C(t)$, the triggering event $E(t)$, and the sequence of actions by $A(t)$.

An event $E \in \Sigma$ is typically represented by $E(\arg_1, \arg_2, \dots, \arg_k)$, where Σ is a set of all events in EFSM. Each event is an external input that may carry a list of arguments. In the EFSM model, states in Q are passive elements; they do not perform any computation or make decisions but rather represent the status of the system at a given point in time. The system execution begins from the initial state (Start) and may terminate at a designated final state by Exit.

The values used in enabling condition and transition actions are stored in a set of variables V . Each action $a_i \in O$ can be classified into one of these types: an assignment action, an output action, or a function call. An assignment action updates the value of variables, an output action produces a value for the external environment, and a function call returns a computer value based on a function $f(v_1, v_2, \dots, v_k)$. When the system is in its initial state $S_b(t)$, the event $E(t)$ occurs, and the enabling condition $C(t)$ evaluates to true, so a transition $t \in T$ is initiated. After the transition is activated, the series of events $A(t)$ is executed, which transitions the EFSM to its terminating state $S_e(t)$ and modifies the system variables. The transition for its related event occurs unconditionally if no enabling condition is specified. The $A(t)$ series of activities is carried out, and the system is moved to the terminating state $S_e(t)$ upon triggering transition t . No additional transition from a state can be linked to $E(t)$ if a transition T is stated at that state without an enabling condition [5,6].

3.1. System Requirements Specification

This section presents five case studies used to evaluate the proposed EFSM-based modeling approach. The selected systems cover various application domains, including banking, security management, online services, e-learning, and embedded control systems. The purpose of these case studies is to demonstrate the applicability and effectiveness of the proposed framework in modeling different types of system behaviors. For each case study, a concise system requirements specification is provided to describe the system functionality in natural language. This is followed by the corresponding EFSM model, which formally represents the system behavior. This structure ensures a consistent transition from informal requirements to formal modeling across all systems. The system requirements specifications for the ATM model are presented below, while such specifications for the other models are provided in Appendix B.

ATM

The ATM system is required to satisfy the following functional requirements:

1. The system shall accept a bank card to initiate a session.
2. The system shall prompt the user to enter a Personal Identification Number (PIN).
3. The system shall validate the entered PIN against the stored PIN.
4. The system shall allow a maximum of three attempts for PIN entry.
5. The system shall display an error message for incorrect PIN entries.
6. The system shall eject the card after three unsuccessful PIN attempts.
7. Upon successful authentication, the system shall allow selection between checking and savings accounts.

8. The system shall support the following transactions for each account:
 - a. Balance inquiry;
 - b. Deposit;
 - c. Withdrawal.
9. The system shall display the account balance upon request.
10. The system shall update the balance after deposit transactions.
11. The system shall allow withdrawals only if sufficient funds are available.
12. The system shall enforce withdrawal constraints, including minimum and maximum limits.
13. The system shall consider ATM cash availability before approving withdrawal transactions.
14. The system shall display appropriate messages for invalid transactions such as insufficient funds or invalid withdrawal limits.
15. The system shall allow the user to print a receipt after transactions.
16. The system shall allow multiple transactions within a session.
17. The system shall terminate the session and eject the card upon user request.

To show how these formal EFSM transition rules work in real life, the next sub-section presents a case study of an ATM system model.

3.2. ATM System Model

EFSM models may be depicted as graphs where states are represented by nodes and transitions by directed edges between states. This section presents a simplified EFSM model of an ATM [5]. This ATM system, as shown in Figure 1 [14], supports three types of transactions: balance inquiry, withdrawal, and deposit, represented by transitions.

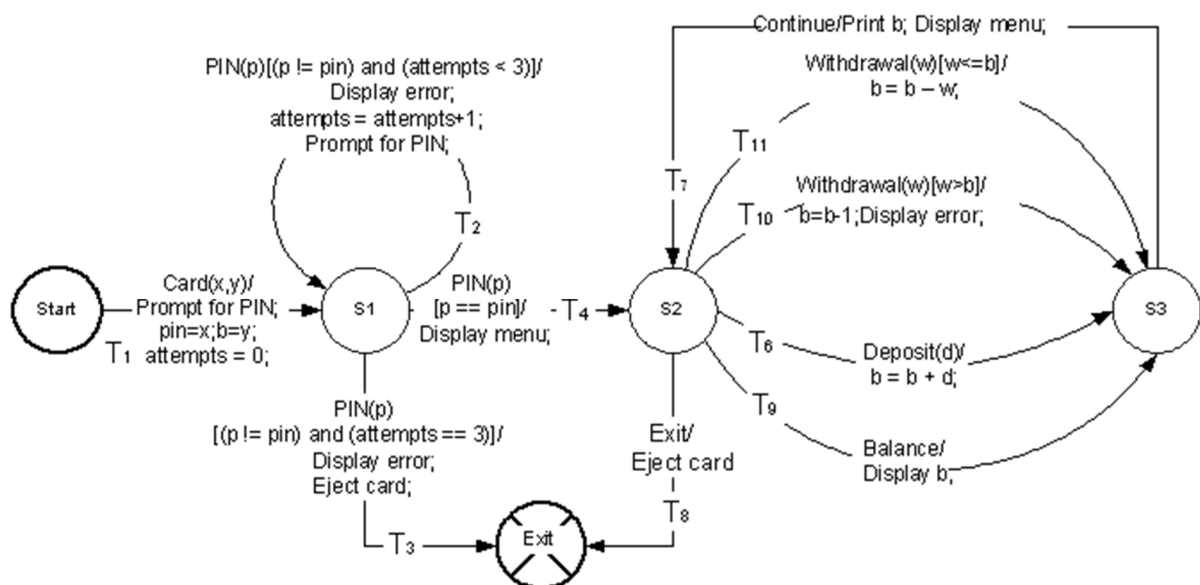


Figure 1. EFSM model of the ATM system.

Before transactions can be performed, an ATM user must enter a valid PIN that is matched against the PIN stored in the ATM card. A user is allowed a maximum of four attempts to enter the valid PIN. For example, the transition labeled T_2 is triggered when the system is in state S_1 , event $PIN(p)$ is received, the value of parameter p does not equal the variable PIN , and the value of variable $attempts$ is less than 3. When the transition is triggered, an error message is displayed, the value of the variable “ $attempts$ ” is incremented, and the user is prompted to enter another PIN. In this example, for transition T_2 , the starting

state is $S_b(T_2) = S_1$, the ending state is $S_e(T_2) = S_1$, the condition is $C(T_2) = (p \neq \text{pin})$ and $(\text{attempts} < 3)$, and the event is $E(T_2) = \text{PIN}(p)$.

3.3. Additional EFSM Models

Other EFSMs were made to show different software systems that highlight various levels of complexity, how states and transitions interact, and how they behave over time, besides the ATM example. To mimic user-driven, transaction-oriented workflows with conditional logic and sequential decision-making, the Food Ordering and Online quiz systems were selected. The Temperature Control and A/C systems were chosen to represent control systems that react to changes and depend on time, characterized by transitions based on feedback, maintaining their state and continuous monitoring. When put together, these systems provide a complete setup for testing AI-generated requirements and creating tests. EFSM diagrams for the four additional systems are provided in Figures A1–A4 in the Appendix A. When combined, these EFSMs offer a thorough testbed for assessing AI-based requirements and test creation. The differences in how complex each model is, how many events they handle, and how they depend on state transitions allow for a fair comparison of the reasoning abilities of ChatGPT, Claude Sonnet, and DeepSeek. The following section describes the approach used to conduct this comparative analysis.

In this article, we assumed that the EFSM model can be run, meaning that it has enough details for the model executor to operate it according to the model's rules (or that the model's rules can be used to create a program that can run the model). Certain actions might not be provided to facilitate model execution; these activities are denoted as “empty” actions. Nevertheless, every action is carried out while the system is being developed. A test is called a series of events with values for the parameters connected to those events.

Consider the following series of events, denoted as t , which includes input values as an example:

$$t = \langle \text{Card} (1234, 100), \text{PIN} (1234), \text{Continue}, \text{Withdrawal} (20), \text{Continue}, \text{Exit} \rangle \quad (1)$$

The following sequence of transitions is triggered when the model shown in Figure 1 is applied to the specified events.

$$\tau(t) = \langle T_1, T_4, T_{11}, T_7, T_8 \rangle \quad (2)$$

Note that no transition occurs for the event “Continue” while in state S_2 ; therefore, there are five transitions in $\tau(t)$ and six events in t .

Let $\tau(t) = \langle T_{i1}, T_{i2}, \dots, T_{ik}, T_{ik+1}, \dots, T_{im} \rangle$ be a series of transitions traversed (executed) when the model M is executed on t . Let $\tau(t)[k]$ be a transition at point k in the sequence $\tau(t)$, meaning that $\tau(t)[k] = T_{ik}$. For instance, $\tau(t)[3] = T_{11}$, which indicates that the transition T_{11} occupies position 3 in the sequence $\tau(t)$, where $\tau(t) = \langle T_1, T_4, T_{11}, T_7, T_8 \rangle$.

For every event $E_i(x_i)$ where $x_i = \arg_1, \arg_2, \dots, \arg_k$ in t , there is only one possible execution of the model M (at most one transition is conducted for a given event $E_i(x_i)$). This is the deterministic assumption of the EFSM model presented in this study.

For a given sequence of events $t = \langle E_1(x_1), E_2(x_2), \dots, E_n(x_n) \rangle$, model M executes a sequence of transitions $\tau(t) = \langle T_{i1}, T_{i2}, \dots, T_{im} \rangle$. Keep in mind that the number of transitions carried out m in $\tau(t)$ does not always equal the number of events n in t .

Figure 2 shows the proposed AI evaluation process using EFSM, illustrating how formal EFSM models are changed into standard prompts, handled by different AI models, and compared to a human engineer's work.

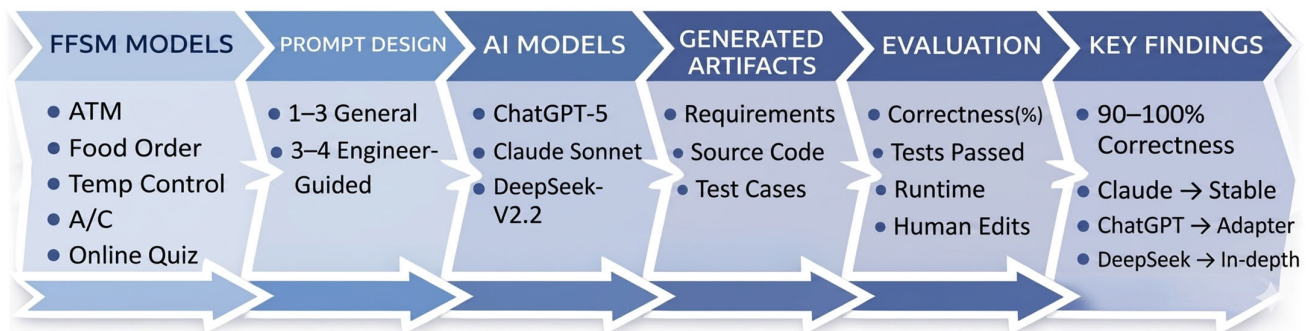


Figure 2. The proposed EFSM-driven AI evaluation pipeline.

4. Methodology

To examine how LLMs can produce software artifacts, specifically system requirements, source code, and unit tests, from formal EFSM-based system models, this study uses an exploratory multi-case methodology. From an early single-case experiment, the technique developed into a systematic multi-model, multi-module comparison framework. It is divided into two linked phases: (1) single-case research that was carried out on an ATM system employing ChatGPT, and (2) a more comprehensive comparative study that assessed three AI models and a human engineer baseline across five EFSM-based systems. Under regulated and repeatable circumstances, this design allows for both quantitative comparison and in-depth qualitative insight.

Figure 3 illustrates the overall framework, presenting the structure and data flow of the proposed approach. The framework integrates formal EFSM models and system constraints, which are transformed into structured prompts through an EFSM interpretation and prompt engineering process. These prompts are then processed by multiple large language models, including ChatGPT-5, Claude Sonnet 4.5, and DeepSeek V3.2, to generate requirement artifacts, test cases, and related outputs. The final outputs are evaluated using multiple quality metrics against a human baseline to assess performance across different models.

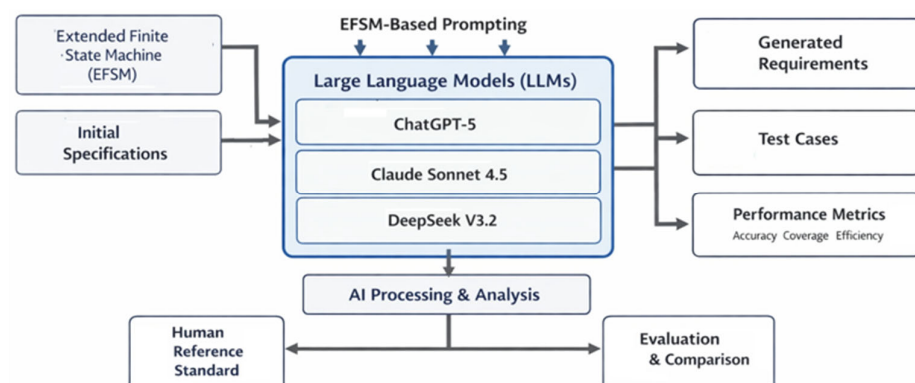


Figure 3. The proposed framework.

4.1. Study Design and Scope

This article follows the evidence-based methodology for case studies in software engineering developed by Pfleeger and colleagues [15]. An exploratory approach was selected because AI-driven requirement and test development are still in their early stages and lack well-established standards. The method allows for close monitoring of LLM behavior, including quick sensitivity, logical reasoning, and artifact correctness while maintaining structured and traceable experimental processes. EFSM modeling was chosen as the base because it has clear rules, predictable outcomes, and can be used in many

types of systems like controllers, transactional services, and interactive applications. EFSM models provide a solid and understandable foundation for the derivation and validation of artifacts created by humans and AI.

The following research questions were addressed to guide the study:

- RQ1: How effectively can AI models extract functional needs from EFSM-based system models?
- RQ2: Can test cases and code produced by AI have the same level of accuracy and coverage as baselines created by humans?
- RQ3: What factors, like prompt structure, model version, or system complexity, most affect AI's performance and reliability?

4.2. Executed Single-Case Study: ATM System (ChatGPT)

The first phase involved an exploratory single-case study focused on ChatGPT's ability to generate software artifacts from an EFSM model of an ATM system. The objective was to observe how effectively the model could interpret structured state-based representations and produce coherent system requirements and documentation.

4.2.1. Case Study Design

The ATM case study was selected because it represents a well-defined transactional system with clear states, transitions, and security constraints. The EFSM model captured operations such as PIN authentication, withdrawals, deposits, and balance inquiries. These characteristics made it suitable for evaluating AI reasoning over both functional and security-critical logic.

4.2.2. Case Study Execution

ChatGPT was used in this phase to generate structured requirements, pseudocode, and API (Application Programming Interface) documentation. Scenario-based analysis was employed to observe its performance in automating documentation and system specification tasks.

4.2.3. Example Scenario: Documentation Generation

- Scenario: Produce API documentation for the transaction process of the ATM.
- Task: Explain every significant operation, such as error handling, authentication, and setup.
- AI Function: ChatGPT generated an API reference template that included methods, input parameters, and anticipated results.
- Result: The documentation produced, consistent with the underlying model, showed a logical structure and took less time to write by hand.

The comparison framework used in the following phase was guided by the case's insights, which enabled the prompt design, assessment criteria, and data collection standards to be improved.

4.3. Comparative Analysis of Multiple Models

The study was expanded into a thorough multi-model comparative analysis after the single-case phase. This stage looked at how well different LLMs and a human engineer performed when creating the same EFSM-based inputs for the same software projects. The ATM System, A/C Controller, Food Order System, Online Quiz System, and Temperature Control System were the five sample modules that were covered by the extended framework. Each module provided a different type of system—transactional, reactive, or interactive—allowing us to see how well AI adapts in different areas.

Four generators were evaluated:

- ChatGPT-5: OpenAI advanced Rational and Synthesis Model [16].
- Claude Sonnet 4.5: Optimized for structured reasoning and long-context stability [17].
- DeepSeek V3.2: Designed for high-speed technical generation with strong logical consistency [18].
- Human Engineer Baseline: A manually implemented reference developed under identical constraints.

Each generator generated (i) functional Python source code, (ii) structured requirements derived from model inputs, and (iii) unit tests covering both valid and invalid behaviors. Every model was given uniform prompts that were created using a single schema that specified the system's context, the necessary outputs, and the validation standards. Only when the generated code failed to execute was there room for minor follow-up clarification; otherwise, the same phrasing was utilized for all runs.

4.3.1. Prompt Template Example

System Description: Vending machine software used was Version 1.0. that accepts coins, checks item stock, and returns change.

Instruction: Generate Python code implementing the system. Include at least three tests covering (1) valid purchase, (2) invalid coin, and (3) out-of-stock selection. Use clear function boundaries and assert outputs for verification.

The prompts were executed in four structured iterations to evaluate performance under increasing complexity.

- Run 1: Baseline EFSM-to-requirements generation.
- Run 2: Integrated requirements-code-test generation.
- Run 3: Robustness testing with boundary and invalid input conditions.
- Run 4: Full state-based specifications with additional logical constraints.

4.3.2. Experimental Controls

- All models used the same prompt wording, without any adaptive rephrasing.
- There was no need for any external tool or Internet access during the generation process.
- Single primary generation run per model, with one optional retry if execution failed.
- All runs were timestamped and logged with metrics, including runtime, accuracy, and test results.

4.3.3. System Model Inputs

All generators received identical system model inputs represented in three complementary formats:

- Textual Requirements: concise feature and constraint summaries.
- Pseudocode/Flow: outlining nominal and exceptional execution paths.
- Simplified UML/EFSM Diagrams: depicting states, transitions, guards, and actions.

Each artifact was version-controlled and harmonized to prevent contradictions between representations.

4.3.4. Human Engineer Baseline

A professional software engineer was tasked with completing the same activities under equivalent time and data constraints. Time-on-task, number of edit iterations, and achieved test pass rates were recorded to establish a performance benchmark for comparison.

4.3.5. Data Collection and Archiving

For each generator and run, the following items were recorded:

- Exact prompt text, parameters, and model version.
- Raw outputs (requirements, code, and tests).
- Execution logs detailing pass/fail results and runtime.
- Post-edit diffs were used to document corrections and effort.
- Operator notes and timestamps.

All outputs were stored in a structured directory layout (`runs/{model}/runN`) to maintain full traceability. The experiments were conducted in a controlled Python 3.13 environment with pinned dependencies. Details of runtime conditions and reproducibility parameters are presented in this section.

4.3.6. Cross-Module Workflow

The same standardized workflow was applied to all five system modules to ensure comparability. Figure 2, introduced earlier, illustrates this workflow, which includes (1) EFSM model preparation, (2) standardized prompt execution across all generators, (3) code and test execution with logging, and (4) evaluation of correctness and completeness metrics.

4.3.7. Data Analysis

Collected outputs were analyzed both quantitatively and qualitatively. Quantitative evaluation measured accuracy, completeness, test coverage, runtime, and human effort. Qualitative analysis examined prompt sensitivity, logical consistency, and reproducibility across models. These analyses provide a complete look at how reliable LLMs are and how they compare to systems created by humans in various EFSM-based applications.

To ensure the correctness of the generated requirements, all outputs were manually validated by a human expert against the corresponding EFSM models. This validation involved verifying alignment with system states, transitions, guard conditions, and actions, ensuring that the generated requirements were semantically consistent with the underlying model.

The proposed pipeline follows a sequential generation process in which requirements, code, and test cases are derived from the same EFSM-based input. While this design reflects a realistic software development workflow, it may introduce dependencies between artifacts, potentially allowing errors to propagate across stages. To partially mitigate this effect, all artifacts were grounded in the same formal EFSM model, which served as a consistent reference for system behavior. The current study relied on EFSM-based grounding and human validation rather than independent oracle-based verification.

5. Experimental Setup

5.1. Workflow

This section describes the controlled experimental setup designed to ensure reproducibility, fairness, and transparency across all evaluated systems and models. Figure 2 showed the same steps taken to create, run, and evaluate the results from each generator, including ChatGPT, Claude Sonnet, DeepSeek, and the human engineer baseline, all using the same EFSM-based method. Each model and the human engineer followed the same procedural flow: EFSM models were first translated into structured textual descriptions and pseudocode. Standardized prompts were then applied to all generators to produce requirements, executable Python code, and corresponding unit tests. Each output was executed using `pytest` to capture correctness, pass rate, and runtime metrics. This uniform workflow ensured a fair and reproducible evaluation across all modules and experimental runs.

5.2. Prompt & Outcome Logging

The output of each generator was recorded in four structured runs (Run 1 to Run 4, as previously described in the Prompt Template Example subsection), each of which increased in complexity to preserve total transparency and traceability. The following data fields were recorded for each run:

Model Version, Runtime (s), Post-Edit Notes, Requirements Generated (number), Tests Generated (number), Tests Passed (x/y), and Pass Rate (%).

Traceability between the raw artifacts and the final performance metrics was ensured by summarizing the execution results in the consolidated results tables for each module (Tables: atm_results—temp_results).

The raw artifacts (such as prompt.md, requirements_run{N}.md, system.py, test_system.py, and pytest_log.txt) were kept in a typical directory structure called runs/{model}/run{N}.

From prompt input to test outcome, this framework offered total traceability.

To ensure fairness, all prompts were rigorously regulated:

- All models had the same language; there were no follow-up clues or rewordings.
- No Internet or external tool access.
- A single one-shot generation was used, and one optional repeat per model.
- Every output was logged, version-tagged, and timestamped for future verification.

Every prompt template followed a set schema that specified the intended outputs, validation criteria, and system environment. To prevent cross-model contamination or environmental intervention, the prompts were carried out one after the other.

5.3. Environment & Reproducibility

All runs were conducted in a controlled Python 3.13 environment under Windows 10 with dependents pinned to ensure reproducibility. The following metadata were recorded for every run:

- We recorded the details of the interpreter and operating system for each run.
- Library versions were stored in a requirements.txt lockfile.
- The deterministic data subsets used a random seed (rng_seed = 42).
- The system stored execution timestamps, hardware specifications, and runtime durations.

No model was allowed to use online search, API calls, or external libraries beyond the predefined environment. Each generator operated entirely using the provided EFSM-based input artifacts.

For the human engineer baseline, reference implementations were selected from verified open-source Python projects corresponding to each system:

- ATM System—Python ATM Interface [19].
- A/C System—Thermostat-FSM [20].
- Food Order System—FoodHub Delivery API implemented using Flask [21].
- Online Quiz System—QuizWiz Python Console Application [22].
- Temperature Control System—BoilerCtrlPID Hydronic Controller [23].

Each baseline was executed under identical runtime conditions and evaluated using the same pytest harness and metric rubric as the AI-generated outputs.

5.4. Reproducibility and Verification

All scripts, prompts, and results have been versioned for public reproducibility and long-term traceability. The consistent setting of the environment, the standardized prompt schema, and the controlled evaluation pipeline ensure that future researchers can replicate every experiment presented in this study. Re-execution under identical hardware

and library configurations consistently yielded equivalent outcomes within a negligible variance margin, validating both the experimental rigor and the repeatability of the presented methodology.

6. Evaluation and Verification

To ensure a standardized assessment across all code generators—ChatGPT, Claude Sonnet, DeepSeek, and the human baseline—a quantitative evaluation framework was adopted.

6.1. Requirements (Number)

This metric represents the number of functional and non-functional requirements generated by each model. It is used as an indicator of how extensively the model captures the EFSM-defined system behavior, reflecting coverage rather than intrinsic quality. In addition to requirement count, correctness was validated through manual inspection by a human expert to ensure alignment with EFSM states, transitions, and guard conditions. This validation step ensures that the generated requirements are semantically consistent with the formal system specification.

6.2. Test (Number)

This metric represents the number of test cases generated and executed for each implementation. It is used to quantify the extent of scenario coverage and behavioral validation derived from the EFSM specification. The generated tests are designed to capture both valid and invalid system behaviors, including state transitions and guard conditions, following a path-oriented exploration of system behavior.

6.3. Correctness (%)

This is computed as $(\text{passed}/\text{total}) \times 100$, representing the proportion of tests passed. This metric serves as an indicator of model accuracy and behavioral conformity to the expected system behavior under EFSM-derived scenarios. However, test pass rate alone does not guarantee full semantic correctness of the generated artifacts. It reflects consistency with the generated test suite rather than complete validation of system behavior. The use of EFSM as a formal reference model, combined with human validation of requirements, provides additional confidence in the semantic validity of the results.

6.4. Run Time (s)

This metric shows the average time (in seconds) required to generate code and execute tests for each run. It is used to assess computational efficiency and responsiveness in comparison to the human engineer baseline.

6.5. Evaluation Scope

Four runs of increasing system complexity, spanning from baseline EFSM to requirements generation to full multi-transactional functioning, were used to test each model. The results were compared against the human engineer baseline in terms of development effort, logical consistency, and behavioral correctness.

6.6. Interpretation

To determine model performance, the evaluation combined runtime efficiency, requirement scope, and correctness. It is important to note that requirement count and test pass rate were used as proxy indicators of coverage and behavioral consistency, rather than complete measures of artifact quality. Furthermore, the evaluation of test cases focused on validating EFSM-derived behavioral scenarios, including state transitions and

guard conditions, following a path-oriented exploration of system behavior rather than maximizing traditional testing metrics such as statement or branch coverage. While established metrics such as coverage and mutation testing could provide additional insight into test effectiveness, they were not included in the current study and are considered part of future work.

In addition, the evaluation did not incorporate independent oracle-based validation or formally verified reference implementations, which could further strengthen the assessment of semantic correctness.

7. Results

This section presents the results of a multi-module evaluation conducted across five EFSM-based systems using three LLMs (ChatGPT, Claude Sonnet, and DeepSeek), alongside a human engineer baseline. All generators were evaluated under identical EFSM-derived specifications and execution conditions to ensure fairness and comparability. Requirement correctness across all models was validated through manual inspection against the EFSM specifications, ensuring alignment with system states, transitions, and guard conditions. While test pass rates provide an indication of behavioral consistency, they do not fully capture semantic correctness. The analysis focused on three primary dimensions: correctness (percentage of passed tests), runtime efficiency, and behavioral stability across iterative prompt refinements. Rather than interpreting results in isolation, this section emphasizes comparative trends observed across systems and models. While detailed per-run metrics are reported in the accompanying tables, the narrative highlights consistent performance patterns, sources of variability, and sensitivity to increasing specification complexity. Together, these results provide an empirical basis for assessing the reliability and scalability of EFSM-grounded LLM-based software generation relative to traditional human-engineered implementations.

7.1. Cross-System Performance Summary

Across all five evaluated systems, the results revealed consistent high-level performance trends shared by the evaluated language models while also exposing meaningful differences in stability, verbosity, and sensitivity to increasing specification constraints. In every system, all three LLMs achieved perfect correctness (100%) in at least one test, showing that using EFSM-grounded prompting can help LLM-generated implementations reach human-level accuracy when the right conditions are met.

Claude Sonnet demonstrated the most overall stability across systems, maintaining consistently high correctness as prompt complexity increased and exhibiting minimal variance between runs.

ChatGPT showed strong adaptability and rapid improvement when prompts were refined but also displayed the greatest volatility, with correctness ranging from complete failure to full accuracy depending on specification strictness and test coverage depth. DeepSeek consistently generated more compact requirement sets and achieved competitive correctness with lower verbosity, though its performance was more sensitive to guard ordering, boundary conditions, and test scaling in later runs. From a runtime point of view, all LLM-based methods ran in less than a second on all systems, and the differences in execution time were not significant compared to the human engineer baseline. The human-made implementations provided consistent and accurate results with quick execution times, but they needed a lot more manual work and had less variety in behavior compared to the AI-generated ones.

In summary, the combined results show that even though no one model was the best in every area, using EFSM-driven LLM generation consistently created correct and usable software. Variations in performance are mainly attributable to prompt structure, constraint

specificity, and model sensitivity rather than fundamental limitations of the EFSM-based approach itself.

7.2. Automated Teller Machine (ATM) System

The ATM system represents a transactional EFSM with authentication logic, guarded operations, and multi-step user workflows, making it a suitable benchmark for evaluating requirement completeness, test coverage depth, and robustness under increasing constraint complexity. The evaluation focused on how effectively each generator captured core banking functionality while preserving correct state transitions and guard enforcement. Under structured EFSM prompting, all models achieved high functional correctness, albeit with notable differences in stability and verbosity. ChatGPT generated the most extensive requirement sets across prompt iterations, demonstrating strong coverage of transactional behavior but also exhibiting substantial variability.

While Run 2 achieved perfect correctness, earlier and later runs revealed sensitivity to expanded robustness testing and guard combinations, resulting in correctness values ranging from 0% to 70.6%. This volatility highlights ChatGPT’s dependence on prompt precision when scaling test depth. Claude Sonnet exhibited the most consistent behavior across ATM runs, maintaining high correctness with moderate requirement growth. Although accuracy declined slightly in the most complex iteration, the model preserved stable guard handling and predictable execution patterns, indicating strong alignment with the EFSM-driven specifications.

DeepSeek created smaller implementations that were still quite accurate, reaching perfect accuracy in the initial tests but becoming more sensitive as the tests expanded, especially in situations focused on robustness. The human engineer baseline served as a deterministic reference, achieving full correctness with minimal runtime while producing the smallest requirement and test set. Although limited in behavioral scope compared to AI-generated outputs, it provided a stable benchmark against which LLM variability and scalability could be assessed.

Tables 1 and 2 show the detailed per-run metrics for requirement counts, test generation, correctness percentages, and execution time. They also show the trade-offs and performance differences between generators and prompt iterations.

Table 1. Human baseline—summary (ATM system).

Field	Value
Source (GitHub)	[19]
Language/Era	Python, human-written (2022)
Requirements Extracted (number)	8 (Deposit, Withdraw, Transfer, Balance, History, Auth, Exit, Passcode Security)
Tests Generated (number)	4
Tests Passed (x/y)	4/4
Correctness (%)	100
Run Time (s)	0.002

Table 2. Per-model results across runs—ATM system.

Model	Run	Requirements (Number)	Test (Number)	Tests Passed (x/y)	Correctness (%)	Run Time (s)
ChatGPT	Run 1	30	6	0/6	0.0	0.16
	Run 2	32	7	7/7	100.0	0.01
	Run 3	39	10	5/10	50.0	0.10
	Run 4	41	17	12/17	70.6	0.09

Table 2. Cont.

Model	Run	Requirements (Number)	Test (Number)	Tests Passed (x/y)	Correctness (%)	Run Time (s)
Claude Sonnet	Run 1	21	16	15/16	93.8	0.21
	Run 2	24	13	13/13	100.0	0.01
	Run 3	35	15	13/15	86.7	0.10
	Run 4	50	17	12/17	70.6	0.09
DeepSeek	Run 1	28	10	10/10	100.0	1.60
	Run 2	14	10	9/10	90.0	0.04
	Run 3	25	12	8/12	66.7	0.09
	Run 4	26	9	6/9	66.7	0.07
Human baseline	-	8	4	4/4	100.0	0.02

7.3. A/C System

The A/C system represents a reactive control EFSM characterized by continuous monitoring, temperature-based guard conditions, and timed state transitions. This system emphasizes precision in boundary handling, prompt sensitivity, and deterministic transition logic, making it well-suited for evaluating how effectively each generator adheres to strict EFSM semantics under increasing specification constraints.

Across models, high correctness was achieved as prompt structure became more explicit, with later runs demonstrating clear improvements in guard enforcement and transition discipline. ChatGPT exhibited strong initial stability, maintaining perfect correctness through multiple iterations before a minor guard misalignment reduced accuracy in the final run. This behavior highlights ChatGPT’s ability to rapidly converge under structured prompting while remaining sensitive to subtle boundary and ordering constraints in complex control logic.

Claude Sonnet demonstrated the strongest overall convergence in the A/C system, achieving complete correctness in the later iterations while expanding requirement coverage in a controlled and human-aligned manner. Its outputs showed consistent handling of temperature thresholds, mode transitions, and EFSM atomicity, indicating robust alignment between prompt structure and generated logic. DeepSeek exhibited the steepest recovery curve, initially struggling with guard ordering and atomic transition enforcement but achieving high correctness once prompt constraints explicitly emphasized EFSM determinism and transition isolation.

The human engineer baseline achieved deterministic behavior with full correctness and stable runtime, serving as a reference implementation for evaluating AI-generated control logic. Despite requiring manual effort, it served as a consistent benchmark for measuring improvements in AI convergence, adaptability, and scalability.

Detailed per-run metrics for requirement extraction, test generation, correctness percentages, and execution time are reported in Tables 3 and 4, which collectively capture the comparative performance and convergence behavior of each generator across prompt iterations.

Table 3. Human baseline—summary (A/C system).

Field	Value
Source (GitHub) latest version	[20]
Language/Era	Python 3 with Tkinter GUI (April 2023)
Requirements Extracted (number)	10 (R1–R10 covering user input, mode switching, and visual feedback)

Table 3. *Cont.*

Field	Value
Tests Generated (number)	10 functional tests across all transitions
Tests Passed (x/y)	10/10
Correctness (%)	100
Run Time (s)	≈0.05

Table 4. Per-model results across runs—A/C system.

Model	Run	Requirements (Number)	Test (Number)	Tests Passed (x/y)	Correctness (%)	Run Time (s)
ChatGPT	Run 1	43	8	8/8	100.0	0.05
	Run 2	40	8	8/8	100.0	0.07
	Run 3	18	11	11/11	100.0	0.08
	Run 4	40	11	10/11	90.9	0.10
Claude Sonnet	Run 1	45	12	8/12	66.7	0.19
	Run 2	30	13	11/13	84.6	0.17
	Run 3	18	11	11/11	100.0	0.09
	Run 4	50	12	12/12	100.0	0.14
DeepSeek	Run 1	15	8	4/8	50.0	0.11
	Run 2	12	8	0/8	0.0	0.12
	Run 3	14	11	10/11	90.9	0.17
	Run 4	13	17	1/17	94.1	0.14
Human baseline	-	10	10	10/10	100.0	0.05

7.4. Food Order System

The food order system represents a multi-transactional EFSM with user authentication, payment handling, and sequential order processing, making it a suitable benchmark for evaluating consistency, message determinism, and scalability under higher interaction complexity. This system emphasizes correct enforcement of transactional ordering, response uniformity, and state-driven control across multiple user scenarios. Across all evaluated models, the food order system exhibited consistently high correctness, with most runs achieving near-perfect or perfect test pass rates. Claude Sonnet demonstrated exceptional stability across all prompt iterations, maintaining 100% correctness throughout while expanding requirements and test coverage in a controlled manner. Its outputs preserved uniform response messages and deterministic transaction handling, closely aligning with the expected EFSM behavior.

ChatGPT also achieved strong performance, reaching perfect correctness in multiple runs while generating more compact requirement sets in later iterations. Minor correctness degradation observed in the final run reflects sensitivity to expanded transactional coverage rather than fundamental modeling errors. DeepSeek produced competitive correctness with comparatively concise implementations, though its performance declined slightly in later runs due to response variability and stricter enforcement of FSM rigidity during complex transaction sequences. The human engineer baseline provided a comprehensive reference implementation with full functional coverage, persistent data handling, and stable correctness. While offering robust and production-grade behavior, its development required substantially greater manual effort, reinforcing the efficiency gains achieved through EFSM-driven LLM-based generation. The detailed metrics for each run, including requirement extraction, test generation, correctness percentages, and execution time, are shown in Tables 5 and 6, which highlight the performance and scalability differences between the generators and prompt iterations.

Table 5. Human baseline—summary (food order system).

Field	Value
Source (GitHub)	[21]
Language/Framework	Python/Flask + PostgreSQL
Requirements Extracted (number)	35
Tests Generated (number)	16
Tests Passed (x/y)	16/16
Correctness (%)	100
Run Time (s)	1.84

Table 6. Per-model results across runs—food order system.

Model	Run	Requirements (Number)	Test (Number)	Tests Passed (x/y)	Correctness (%)	Run Time (s)
ChatGPT	Run 1	31	7	7/7	100.0	0.02
	Run 2	37	8	8/8	100.0	0.03
	Run 3	20	13	13/13	100.0	0.43
	Run 4	20	10	9/10	90.0	0.57
Claude Sonnet	Run 1	33	12	12/12	100.0	0.04
	Run 2	46	13	13/13	100.0	0.03
	Run 3	55	28	28/28	100.0	0.43
	Run 4	80	25	25/25	100.0	0.68
DeepSeek	Run 1	35	38	38/38	100.0	0.12
	Run 2	33	25	25/25	100.0	0.20
	Run 3	20	15	14/15	93.0	0.45
	Run 4	20	15	14/15	93.0	0.49
Human baseline	-	35	16	16/16	100.0	1.84

7.5. Online Quiz System

The online quiz system represents a highly deterministic EFSM characterized by explicit state sequencing, timed guards, and strict validation of permissible actions. This system serves as a strong benchmark for evaluating determinism, reproducibility, and alignment with formal EFSM semantics, particularly under iterative prompt refinement and robustness-focused testing. Across all models, the online quiz system exhibited the highest overall consistency among the evaluated modules. Incremental prompt refinement led to steady improvements in discipline and correctness, with later runs demonstrating strong enforcement of activation windows, submission constraints, and error handling.

ChatGPT generated the most comprehensive requirement sets, capturing a wide range of edge cases and recovery behaviors. While increased verbosity occasionally introduced minor sensitivity to boundary conditions, correctness remained consistently high across runs.

Claude Sonnet demonstrated exceptional stability and determinism, achieving near-perfect or perfect correctness in most iterations with minimal variance. Its outputs closely mirrored human-engineered logic when prompts balanced clarity and scope, maintaining precise guard enforcement and predictable state transitions.

DeepSeek produced leaner implementations with competitive correctness, achieving flawless execution in constrained deterministic scenarios while showing brief regressions when temporal boundary conditions were emphasized more aggressively.

The human engineer baseline provided a deterministic reference implementation with strong alignment to EFSM principles and consistent execution behavior.

Although limited in scope compared to AI-generated outputs, it established a stable correctness benchmark against which the reproducibility and scalability of LLM-based generation could be assessed.

Detailed per-run metrics for requirement counts, test generation, correctness percentages, and execution time are reported in Tables 7 and 8, which collectively illustrate the convergence behavior, determinism, and comparative performance across generators and prompt iterations.

Table 7. Human baseline—summary (online quiz system).

Field	Value
Source (GitHub)	[22]
Language/Era	Python 3 (console-based, object-oriented)
Requirements Extracted (number)	12 core requirements (R1–R12)
Tests Generated (number)	11 (atomic, pytest-style)
Tests Passed (x/y)	10/11
Correctness (%)	90.9
Run Time (s)	0.06 (single-threaded local execution)

Table 8. Per-model results across runs—online quiz system.

Model	Run	Requirements (Number)	Test (Number)	Tests Passed (x/y)	Correctness (%)	Run Time (s)
ChatGPT	Run 1	25	12	12/12	100.0	0.05
	Run 2	30	16	15/12	93.8	0.19
	Run 3	76	18	17/18	94.4	0.16
	Run 4	35	20	19/20	95.0	0.24
Claude Sonnet	Run 1	30	13	12/13	92.3	0.04
	Run 2	30	16	16/16	100.0	0.12
	Run 3	75	26	26/26	100.0	0.12
	Run 4	35	27	26/27	96.3	0.08
DeepSeek	Run 1	17	12	12/12	100.0	0.06
	Run 2	20	16	16/16	100.0	0.10
	Run 3	22	18	16/18	88.9	0.34
	Run 4	25	18	18/18	100.0	3.03
Human baseline	-	12	11	10/11	90.9	0.06

7.6. Temperature Control System

The temperature control system represents a time-driven EFSM with cyclic execution, safety thresholds, and multi-range guard validation, making it a rigorous benchmark for evaluating temporal consistency, boundary handling, and scalability under prolonged execution cycles. This system emphasizes precise enforcement of temperature limits, timer resets, and stable-state transitions across repeated control loops.

Across all evaluated models, high functional correctness was achieved as prompt constraints increasingly emphasized EFSM determinism and explicit guard definitions. Claude Sonnet demonstrated the strongest overall stability in this system, maintaining perfect correctness across all prompt iterations while expanding requirement coverage substantially. Its outputs consistently preserved safe temperature ranges, accurate timer handling, and predictable state transitions, even under extended simulation cycles. ChatGPT achieved flawless correctness in early and intermediate runs but exhibited sensitivity in the final iteration when additional constraints increased guard strictness and state validation complexity. This behavior highlights ChatGPT’s responsiveness to prompt structure

while emphasizing the importance of precise boundary specification in time-dependent control systems. DeepSeek produced compact and generally accurate implementations, with minor regressions emerging in later runs related to timeout synchronization and return-message consistency rather than core EFSM violations. The human engineer baseline provided a deterministic reference with stable runtime and full correctness across all tests. While offering robust continuous control behavior, it required manual configuration and domain-specific tuning, reinforcing the efficiency and adaptability benefits of EFSM-driven LLM-based generation.

Detailed per-run metrics for requirement extraction, test generation, correctness percentages, and execution time are reported in Tables 9 and 10, which summarize the comparative performance, stability, and scalability across generators and prompt iterations.

Table 9. Human baseline—summary (temperature control system).

Field	Value
Source (GitHub)	[23]
Language/Framework	Python/PID Hydronic Controller
Requirements Extracted (number)	70
Tests Generated (number)	19
Tests Passed (x/y)	19/19
Correctness (%)	100
Run Time (s)	0.08

Table 10. Per-model results across runs—temperature control system.

Model	Run	Requirements (Number)	Test (Number)	Tests Passed (x/y)	Correctness (%)	Run Time (s)
ChatGPT	Run 1	16	8	8/8	100.0	0.03
	Run 2	20	8	8/8	100.0	0.06
	Run 3	14	9	9/9	100.0	0.10
	Run 4	18	8	5/8	62.0	0.20
Claude Sonnet	Run 1	22	18	18/18	100.0	0.09
	Run 2	40	18	18/18	100.0	0.11
	Run 3	80	10	10/10	100.0	0.08
	Run 4	110	26	26/26	100.0	0.29
DeepSeek	Run 1	16	16	16/16	100.0	0.04
	Run 2	16	17	17/17	100.0	0.14
	Run 3	20	37	35/37	94.0	0.36
	Run 4	20	34	29/34	85.0	0.27
Human baseline	-	70	19	19/19	100.0	0.08

Overall, the results across all five systems demonstrate that EFSM-grounded prompting enables LLMs to generate correct, executable, and testable software artifacts with performance comparable to, and in some cases exceeding, the human engineer baseline. While differences in stability, verbosity, and sensitivity to prompt constraints were observed across models, correctness consistently improved as EFSM structure and specification clarity increased. These findings indicate that performance variability is primarily driven by prompt design and constraint articulation rather than limitations of the EFSM-based generation approach itself. The implications of these results, along with their impact on scalability, reliability, and practical software engineering workflows, are discussed in the following section.

7.7. Cross-System Comparative Analysis

Along with the evaluations of each system, a comparison was made to look at the overall accuracy and speed of all of the tested systems (Figures 4 and 5). This analysis did not look at individual runs; instead, it combined performance metrics to show the overall trends in model reliability, efficiency, and stability. This provides a more complete picture of how different models behave. The figures show that the human engineer baseline achieved the highest overall correctness, while Claude Sonnet demonstrated the strongest average correctness among the evaluated LLMs, followed by ChatGPT and DeepSeek.

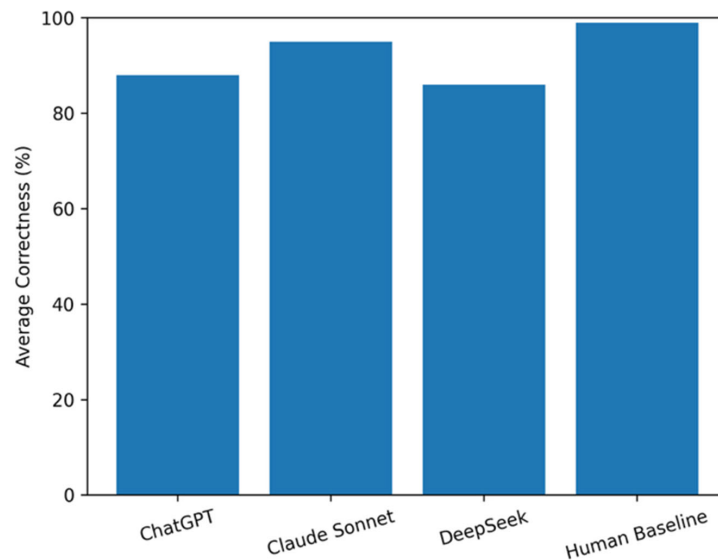


Figure 4. Average correctness across models (all systems).

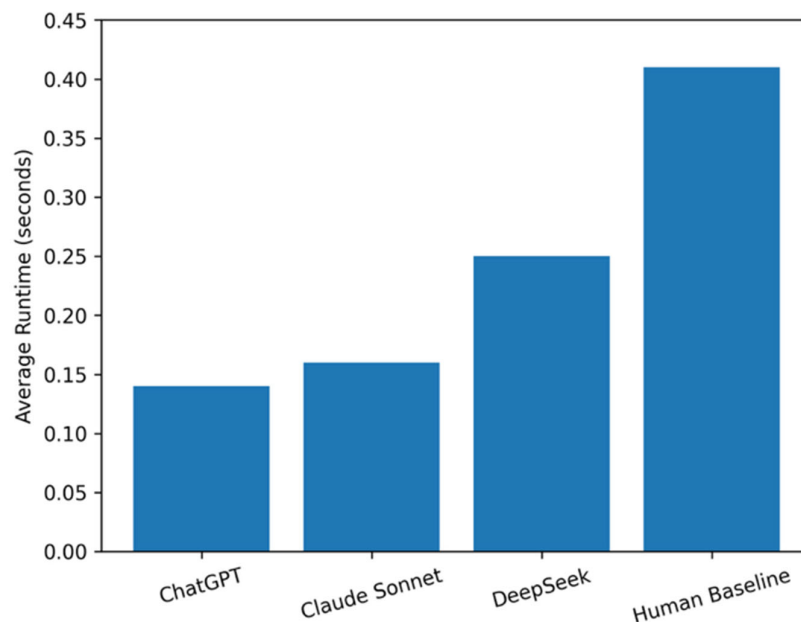


Figure 5. Runtime efficiency comparison across models.

8. Discussions

In this discussion, we integrate the behaviors of different models and assess the overall performance trends of the systems, using the comparison framework from Section 3 and the results from Section 7. The evaluation of several models revealed that they acted similarly across all five EFSM-based systems (ATM, A/C controller, food order, online quiz, and temperature control), as shown in tables Tables 2, 4, 6, 8 and 10, and Figures 4 and 5.

Each model showed unique strengths that matched its rapid sensitivity and reasoning structure. In every run, Claude Sonnet maintained near-perfect correctness despite producing the most extensive and scalable requirement sets, up to 80 requirements in the food order system and 110 requirements in the temperature control module. ChatGPT showed outstanding flexibility and speed, generating accurate and test-passing systems in milliseconds. It performed especially well in mid-stage runs, like the online quiz and A/C systems, where it consistently obtained 100% pass rates. Later, in very limited tests, like temperature control Run 4, where flexibility was tightly controlled, its results still showed some variation. Particularly in earlier iterations of the food and ATM systems, DeepSeek demonstrated remarkable runtime efficiency and wider test coverage.

However, as complexity increased in subsequent iterations, its accuracy somewhat decreased, suggesting susceptibility to multi-layered EFSM dependencies. All modules' human baselines were taken from GitHub implementations that were already in place and represented human-engineered systems created using conventional development techniques.

Specifically, in all case studies, the human role corresponded to the design and implementation of the baseline systems (including coding, logic design, and verification), as well as the definition of the original system requirements in the referenced GitHub implementations. In contrast, the AI models (ChatGPT and DeepSeek) were only responsible for generating EFSM-based implementations from formally specified models and were not involved in the initial system design or baseline development. The testing phase was conducted uniformly across both human and AI-generated implementations to ensure a fair comparison.

As a benchmark for logical consistency, interpretability, and structural completeness, each baseline maintained 100% accuracy. These systems' initial development, however, necessitated a significant amount of human design, debugging, and verification work, underscoring the time and cost benefits of AI-driven generation. In comparison, the huge language models achieved full test coverage and comparable soundness through automated synthesis, producing operational equivalents in a matter of seconds. These findings show a way to combine human and AI efforts in software engineering, where human engineers are essential for checking specific rules, handling unusual situations, and ensuring that designs meet real-world needs, while large language models can efficiently create the written and structural parts of system design. Claude Sonnet was the most reliable and comprehensive generator overall, striking a balance between scalability, precision, and logical coherence across various domains. While DeepSeek's result illustrates the importance of stringent prompt alignment for precision, ChatGPT is still a highly flexible model that excels in efficiency and requirement clarity. The results show that using organized EFSM prompting helps make large language models reliable partners in formal software development by enabling the creation of systems that are easy to reproduce, track, and scale.

9. Threats to Validity

When interpreting the results, it is important to consider the limitations of this study. First, the behavior of large language models can be sensitive to how prompts are written. Although standardized prompts were used across all models to ensure fairness, small differences in interpretation may still influence the generated outputs. Second, the evaluation metrics used in this study, such as the number of generated requirements, number of test cases, and test pass rate, primarily reflect coverage and behavioral consistency rather than fully capturing artifact quality. While requirement correctness was manually validated by a human expert against the EFSM models, other important aspects such as ambiguity, redundancy, and traceability were not quantitatively measured. Similarly, test effectiveness was evaluated based on execution results, without incorporating more advanced metrics such

as code coverage or mutation testing. In addition, the generated test cases focused on validating EFSM-derived behavioral scenarios and may not have fully captured fault detection capability or discriminative strength when compared to traditional testing approaches.

Another limitation is that the study was conducted on five EFSM-based systems, which, although diverse, may not fully represent larger or more complex real-world applications. In addition, the pipeline follows a sequential process in which requirements, code, and tests are generated together, which may allow errors to propagate across artifacts despite the use of EFSM as a formal reference. Furthermore, the evaluation did not include independent oracle-based validation or formally verified reference implementations, which may limit the ability to fully assess semantic correctness. Finally, although the experiments were performed in a controlled environment, large language models inherently produce non-deterministic outputs, which may lead to slight variations in results across runs.

Future work will address these limitations by incorporating more comprehensive requirement quality metrics, such as ambiguity and traceability analysis, integrating advanced testing measures including coverage and mutation testing, introducing independent validation mechanisms for each generated artifact, and extending the framework to larger and more complex systems as well as real-world case studies.

10. Conclusions and Future Works

The journey from formal system models to comprehensive software requirements, source code, and test cases can be efficiently accelerated and structured by using big language models, as this work shows. Claude Sonnet 4.5 and DeepSeek V3.2 were tested on five systems based on EFSM, which included the ATM, A/C, Food Order, Online Quiz, and Temperature Control systems. The evaluation was conducted against a human engineer baseline. The suggested structure preserved full traceability to the original models while enhancing documentation quality, efficiency, and consistency. When guided by organized EFSM prompts and tested in consistent settings, the results show that AI models can achieve accuracy and coverage that is close to human levels, and sometimes even better. However, AI works best when combined with thorough human evaluation. Engineers provide contextual awareness, domain judgment, and accountability, which AI systems lack, ensuring compliance with regulations and practical feasibility. This emphasizes the benefits of a collaboration between humans and AI rather than complete automation. AI-driven modeling and test generation can help boost productivity, quality, and traceability while ensuring safety, security, and ethical standards in modern software engineering, as long as it is properly managed and included in existing development processes. Based on the observed results, AI-driven modeling and test generation can significantly enhance productivity, quality, and traceability. At the same time, the results highlight the necessity of maintaining human oversight to ensure safety, security, and adherence to ethical standards. Therefore, the conclusions drawn are directly supported by the experimental findings, emphasizing that the successful adoption of AI in modern software engineering depends on its careful integration within existing development processes rather than its standalone use.

Several research extensions and useful directions for developing AI-assisted software engineering are revealed by building upon the finished multi-model evaluation that was reported in this article. A strong basis for investigating larger system types, more intricate AI-human collaboration mechanisms, and more stringent long-term validation frameworks is provided by the reproducibility in five EFSM-based systems.

Future paths include

- To preserve semantic consistency across representations, extend AI procedures to analyze and reconcile multi-view modeling artifacts (EFSM, Statecharts, sequence, and activity diagrams).

- Patterns for SDLC integration: Integrate the AI tools under evaluation into Waterfall, DevOps, and Agile pipelines; for example, continuous integration jobs that automatically build and validate test cases and requirements from changing models.
- Cross-domain replication: Evaluate domain-specific safety and compliance restrictions by applying the framework to regulated industries like healthcare, automobile, and aerospace.
- Human–AI collaboration frameworks: Involve people in the learning process to improve models, and set clear roles for reviewers, approval steps, and feedback criteria to ensure clarity, thoroughness, and tracking
- Long-term field studies: Track how productivity improves over time, how defects decrease, and how well AI-generated items meet various standards through several phases of testing.
- To meet new software safety standards, create guidelines that cover how to explain AI decisions, avoid bias in models, protect sensitive data, and ensure that AI-generated materials can be audited.
- Education and upskilling: Include AI-assisted requirement and test creation with a focus on responsible adoption, validation, and verification in professional training programs and software engineering curricula.

To make it easier to compare results in schools and businesses, future research will focus on broadening this proven framework to include different types of models and tools, providing clear performance standards for how well AI and humans work together, and sharing free resources.

Author Contributions: Conceptualization, Z.S. and L.T.; methodology, Z.S. and L.T.; software, Z.S., Y.H. and N.T.; validation, Z.S., Y.H. and N.T.; formal analysis, Z.S., Y.H. and N.T.; investigation, Z.S., Y.H. and N.T.; resources, Z.S.; data curation, Z.S., Y.H. and N.T.; writing—original draft preparation, Z.S., Y.H. and N.T.; writing—review and editing, Z.S. and L.T.; visualization, Z.S.; supervision, Z.S.; project administration, Z.S.; funding acquisition, Z.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: No new data were created or analyzed in this study.

Acknowledgments: The authors gratefully acknowledge the Gulf University for Science and Technology (GUST), Kuwait, for covering the publication fees of this article.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AI	Artificial Intelligence
LLM	Large Language Model
EFSM	Extended Finite State Machine
SDLC	Software Development Life Cycle
ATM	Automated Teller Machine
A/C	Air Conditioning
API	Application Programming Interface
PIN	Personal Identifier Number
UML	Unified Modeling Language

Appendix A. Extended Finite State Machine (EFSM) Models Used in the Study

This appendix presents the Extended Finite State Machine (EFSM) models used in this study to evaluate and illustrate the proposed AI-driven approach for system requirements analysis and automated test case generation.

These models represent different application domains and are used as reference examples to demonstrate how system behavior can be modeled using EFSM representations.

Figure A1 shows the EFSM model of a food order system, which represents the workflow of placing and processing orders in an online food ordering environment. The model captures key states such as order placement, payment processing, and order confirmation.

Figure A2 illustrates the EFSM model of an online quiz system. This model represents the process of user authentication, quiz initialization, question navigation, answer submission, and result generation.

Figure A3 presents the EFSM model of a temperature control system, which demonstrates how environmental conditions are monitored and how control actions are triggered to maintain the desired temperature range.

Figure A4 shows the EFSM model of an air-conditioning system, highlighting the transitions between operational states such as cooling, standby, and shutdown depending on system inputs and environmental parameters.

These EFSM models were adapted from previously published sources and open repositories and are included in this appendix to provide additional clarity on the structure of the systems used in the experimental analysis.

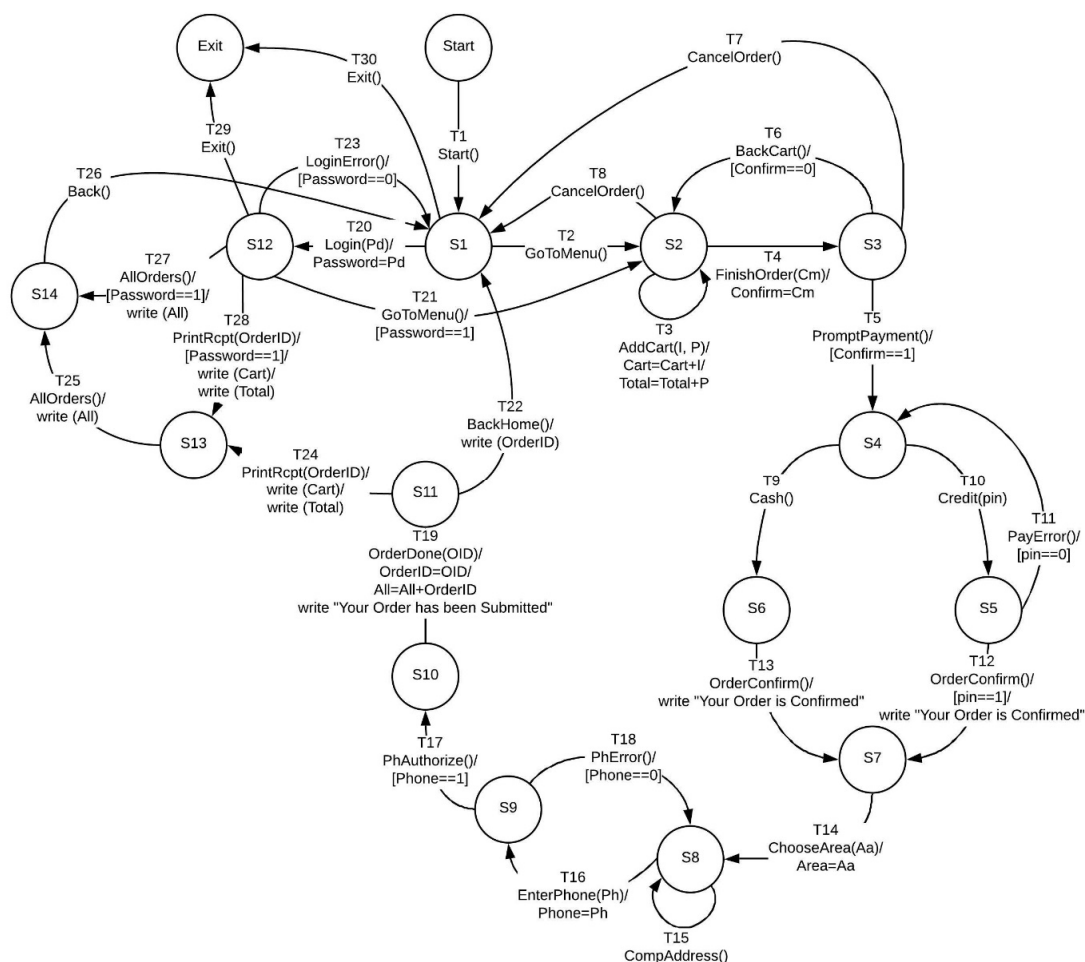


Figure A1. EFSM model of the food order system (adapted from University of Maryland, 2004 [24]).

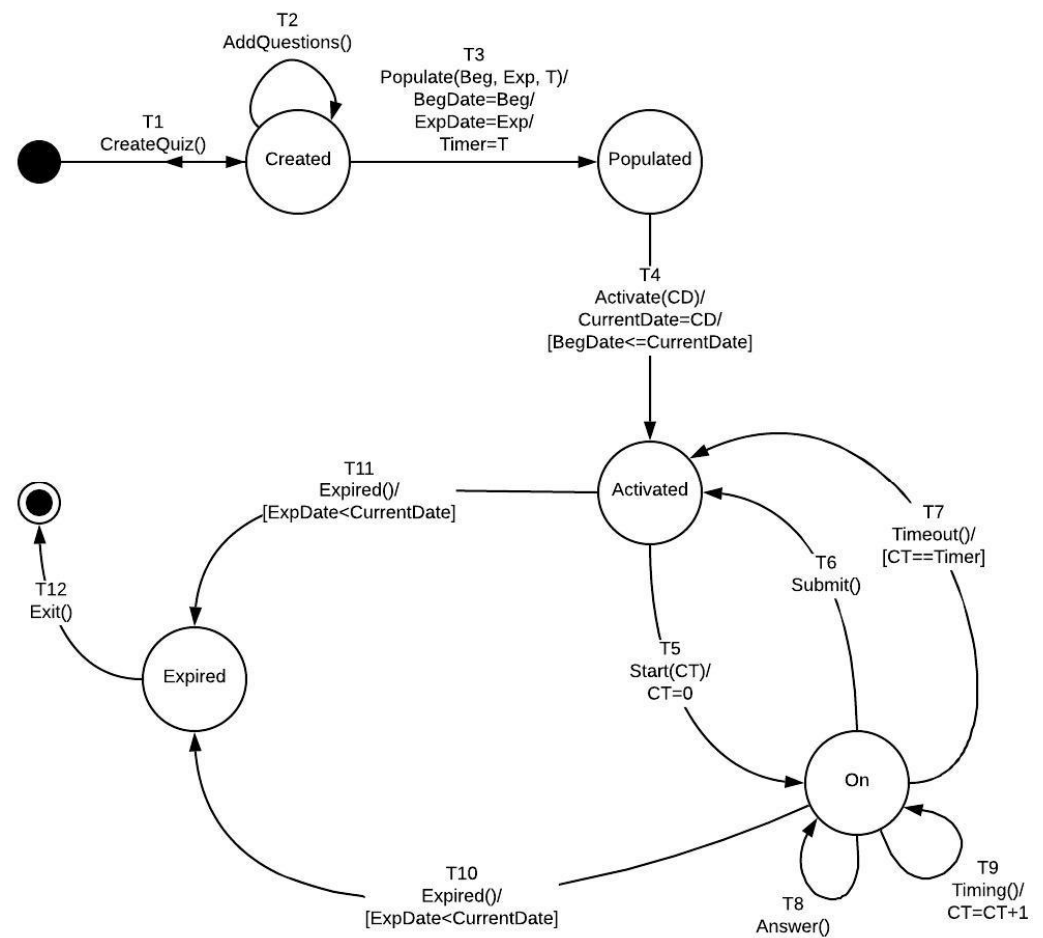


Figure A2. EFSM model of an online quiz system (adapted from Comathuna, GitHub [14]). The solid black circle indicates the Initial Pseudostate (starting point), while the bullseye symbol represents the Final State (completion).

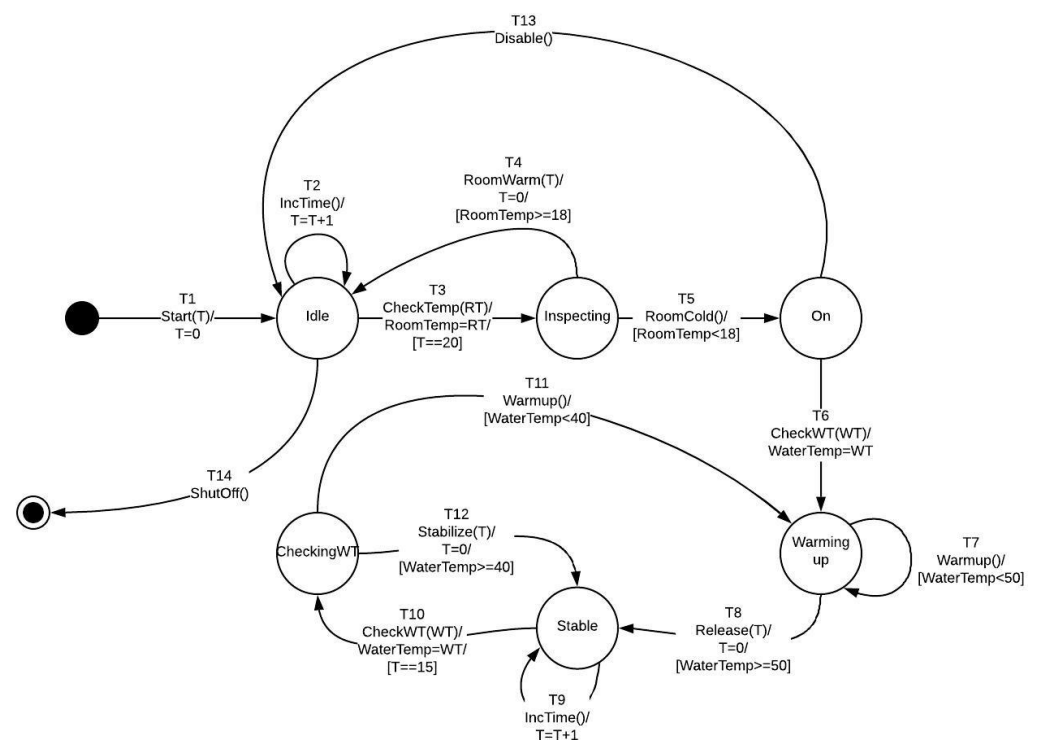


Figure A3. EFSM model of the temperature control system (adapted from [25]).

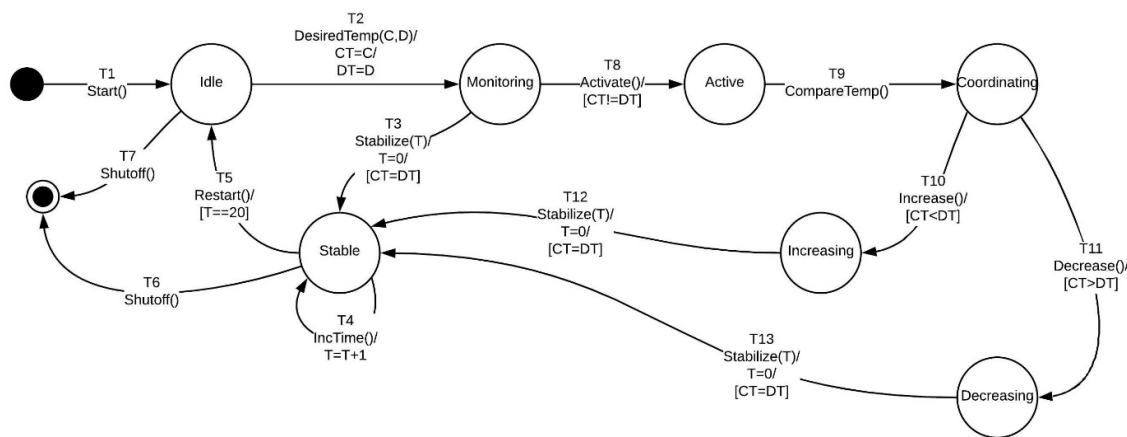


Figure A4. EFSM model of an air-conditioning system (adapted from [25]).

Appendix B. Systems Requirements Specification-Other Models

Appendix B.1. A/C System

The following requirements describe the functional behavior of the (A/C) system. The system is designed to regulate room temperature by comparing the current temperature with the desired temperature and adjusting accordingly.

1. The system shall initialize in an idle state.
2. The system shall accept a desired temperature input from the user.
3. The system shall continuously monitor the current room temperature.
4. The system shall compare the current temperature with the desired temperature.
5. The system shall activate when the current temperature is not equal to the desired temperature.
6. The system shall increase or decrease the temperature depending on the comparison result.
7. The system shall stop adjusting when the current temperature equals the desired temperature.
8. The system shall enter a stable state when both temperatures are equal.
9. The system shall periodically (every 20 min) recheck the temperature conditions.
10. The system shall reactivate if a temperature mismatch is detected during periodic checks.
11. The system shall remain in a stable state if no temperature change is required.
12. The system shall allow shutdown from either idle or stable states.
13. The system shall reset timing counters after stabilization.

Appendix B.2. Food Ordering System

The following requirements describe the functional behavior of the online food ordering system. The system supports both guest users and registered users, allowing them to browse menus, place orders, and manage transactions.

1. The system shall allow users to start a session as either a guest or a registered user.
2. The system shall allow registered users to log in using valid credentials.
3. The system shall display an error message for invalid login attempts.
4. The system shall allow users to access the menu and browse available items.
5. The system shall allow users to add items to a shopping cart.
6. The system shall maintain a running total of selected items.
7. The system shall allow users to review, modify, or cancel their order before confirmation.
8. The system shall allow users to finalize the order or return to the cart to add more items.
9. The system shall prompt the user for payment upon order confirmation.

10. The system shall support payment via cash or credit card.
11. The system shall validate credit card payment before confirming the order.
12. The system shall display an error message for invalid payment attempts.
13. The system shall confirm the order after successful payment.
14. The system shall require users to select a delivery area.
15. The system shall allow users to enter and validate a phone number.
16. The system shall generate a unique order ID upon order completion.
17. The system shall allow users to view order details and print receipts.
18. The system shall allow registered users to view all previous orders.
19. The system shall allow navigation back to the home interface.
20. The system shall allow users to exit the system at any stage.

Appendix B.3. Online Quiz System

The following requirements describe the functional behavior of the online quiz system. The system supports quiz creation, scheduling, activation, and user participation within defined time constraints.

1. The system shall allow the creation of a new quiz.
2. The system shall allow adding questions to the quiz during the creation phase.
3. The system shall allow setting quiz parameters, including start date, expiry date, and duration.
4. The system shall store the quiz configuration before activation.
5. The system shall activate the quiz when the current date reaches or exceeds the defined start date.
6. The system shall allow users to begin the quiz once it is activated.
7. The system shall start a timer when the quiz begins.
8. The system shall allow users to submit the quiz before the timer expires.
9. The system shall automatically submit or terminate the quiz when the time limit is reached.
10. The system shall allow users to answer questions during the active quiz session.
11. The system shall continuously track elapsed time during the quiz.
12. The system shall mark the quiz as expired when the current date exceeds the expiry date.
13. The system shall prevent access to expired quizzes.
14. The system shall allow the system to terminate the quiz session upon exit.

Appendix B.4. Temperature Control System

The following requirements describe the functional behavior of the temperature control system. The system is designed to maintain a comfortable room temperature by regulating heat through water-based heating pipes.

1. The system shall initialize in an idle state.
2. The system shall periodically (every 20 min) check the room temperature.
3. The system shall compare the room temperature against a predefined threshold (18 °C).
4. The system shall remain idle if the room temperature is greater than or equal to the threshold.
5. The system shall activate if the room temperature is below the threshold.
6. The system shall monitor the water temperature used for heating.
7. The system shall increase the water temperature if it is below 50 °C.
8. The system shall release heated water into the pipes once the required temperature is reached.

9. The system shall periodically (every 15 min) check the water temperature during operation.
10. The system shall reheat the water if the temperature drops below 40 °C.
11. The system shall maintain a stable state if the water temperature is sufficient (≥ 40 °C).
12. The system shall allow the system to be disabled while it is active.
13. The system shall allow shutdown only when the system is in the idle state.
14. The system shall reset timing counters after each monitoring cycle.

References

1. Ahmad, K.; Abdelrazek, M.; Arora, C.; Bano, M.; Grundy, J. Requirements engineering for artificial intelligence systems: A systematic mapping study. *Inf. Softw. Technol.* **2023**, *158*, 107176. [\[CrossRef\]](#)
2. Arora, C.; Aydemir, F.B.; Frattini, J. AIRE 2024: 11th International Workshop on Artificial Intelligence and Requirements Engineering. In Proceedings of the 2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW), Reykjavik, Iceland, 24–25 June 2024; pp. 1–2.
3. Felderer, M.; Enou, E.P.; Tahvili, S. Artificial intelligence techniques in system testing. In *Optimising the Software Development Process with Artificial Intelligence*; Springer Nature: Singapore, 2023; pp. 221–240.
4. Mehraj, A.; Zhang, Z.; Systä, K. A tertiary study on artificial intelligence for requirements engineering. In *Proceedings of the International Working Conference on Requirements Engineering: Foundation for Software Quality*; Springer: Cham, Switzerland, 2024; pp. 159–177.
5. Tahat, L.; Korel, B.; Koutsogiannakis, G.; Almasri, N. State-based models in regression test suite prioritization. *Softw. Qual. J.* **2017**, *25*, 703–742. [\[CrossRef\]](#)
6. Duale, A.Y.; Uyar, M.U. A method enabling feasible conformance test sequence generation for EFSM models. *IEEE Trans. Comput.* **2004**, *53*, 614–627. [\[CrossRef\]](#)
7. Guo, Q.; Cao, J.; Xie, X.; Liu, S.; Li, X.; Chen, B.; Peng, X. Exploring the potential of ChatGPT in automated code refinement: An empirical study. In Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE), Lisbon, Portugal, 14–20 April 2024; pp. 1–13.
8. Kelley, K. Automated test case generation from correct and complete system requirements models. In Proceedings of the 2009 IEEE Aerospace Conference, Big Sky, MT, USA, 4–11 April 2009; pp. 1–10.
9. Rodrigues, A.; Vilela, J.; Silva, C. A systematic mapping study on techniques for generating test cases from requirements. In *Proceedings of the 9th International Conference on Internet of Things, Big Data and Security (IoTBDs 2024)*, Madeira, Portugal, 3–5 April 2024; SCITEPRESS: Setúbal, Portugal, 2024; pp. 123–134.
10. Bhalla, J.S.; Jodhka, M.K. Leveraging large language models in the software development lifecycle: Opportunities and challenges. *Int. J. Adv. Comput. Sci. Appl.* **2025**, *16*, 11. [\[CrossRef\]](#)
11. Nguyen-Duc, A.; Cabrero-Daniel, B.; Przybylek, A.; Arora, C.; Khanna, D.; Herda, T.; Rafiq, U.; Melegati, J.; Guerra, E.; Saari, M.; et al. Generative artificial intelligence for software engineering—A research agenda. *Softw. Pract. Exp.* **2025**, *55*, 1806–1843. [\[CrossRef\]](#)
12. Al-Ahmad, A.; Kahtan, H.; Tahat, L.; Tahat, T. Enhancing software engineering with AI: Key insights from ChatGPT. In Proceedings of the 2024 International Conference on Decision Aid Sciences and Applications (DASA), Manama, Bahrain, 23–25 February 2024; pp. 1–5.
13. Fonseca i Casas, P. Using specification and description language to formalize multiagent systems. *Appl. Artif. Intell.* **2014**, *28*, 504–531. [\[CrossRef\]](#)
14. Korel, B.; Singh, I.; Tahat, L.; Vaysburg, B. Slicing of state-based models. In Proceedings of the International Conference on Software Maintenance (ICSM 2003), Amsterdam, The Netherlands, 22–26 September 2003; pp. 34–43.
15. Kitchenham, B.A.; Dyba, T.; Jørgensen, M. Evidence-based software engineering. In Proceedings of the 26th International Conference on Software Engineering (ICSE 2004), Edinburgh, UK, 23–28 May 2004; pp. 273–281.
16. OpenAI. ChatGPT (GPT-5.2). Available online: <https://chat.openai.com/> (accessed on 17 April 2026).
17. Anthropic. Claude (Sonnet 4.5). Available online: <https://www.anthropic.com/> (accessed on 17 April 2026).
18. DeepSeek. DeepSeek (V3.2). Available online: <https://www.deepseek.com/> (accessed on 17 April 2026).
19. JEETAHIRWAR. Python ATM Interface. Available online: https://github.com/JEETAHIRWAR/Python_ATM_Interface (accessed on 17 April 2026).
20. Khandelwal, A. Thermostat FSM. Available online: <https://github.com/Khandelwal05/Thermostat-FSM> (accessed on 17 April 2026).
21. Kodek-Sleuth. FoodHub Delivery API. Available online: <https://github.com/kodek-sleuth/FoodHub-Delivery-API/> (accessed on 17 April 2026).

22. AJBrohi. QuizWiz-Py. Available online: <https://github.com/AJBrohi/quizwiz-py> (accessed on 17 April 2026).
23. Snyder, H. Hydronic Simulation. Available online: <https://github.com/hsnyder/hydronic-simulation> (accessed on 17 April 2026).
24. University of Maryland. Automated Food Ordering System. Available online: <https://eng.umd.edu/~austin/ense621.d/projects04.d/project-food-ordering.html> (accessed on 17 April 2026).
25. Wang, Y.; Li, Z.; Zhao, R. Dependence-based model healing. In Proceedings of the IEEE 39th Annual Computer Software and Applications Conference (COMPSAC 2015), Taichung, Taiwan, 29 July–2 August 2015; pp. 556–561.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.